Análisis de algoritmos

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

3 de marzo de 2026

Algorithm Analysis

Conceptos fundamentales para el análisis de algoritmos y notación asintótica.

Universidad Pontificia Bolivariana

Índice

1. Análisis de Algoritmos **2**
1.1. Eficiencia de algoritmos . . . 2
1.2. Descripción de la eficiencia por medio de un modelo analítico . . . 2
1.3. Metodología experimental para evaluar la eficiencia . . . 3
1.3.1. Evaluación experimental del tiempo de ejecución . . . 3
1.3.2. Evaluación experimental del consumo de memoria . . . 4
1.4. Metodología analítica . . . 4
1.4.1. Modelos de costo . . . 6
1.4.2. Notación asintótica . . . 8
1.5. Análisis de casos . . . 9
1.6. Análisis de espacio . . . 11

1. Análisis de Algoritmos

1.1. Eficiencia de algoritmos

La eficiencia de un algoritmo es una medida de la cantidad de recursos computacionales que consume al resolver un problema. Los dos recursos fundamentales son el **tiempo de ejecución** y el **espacio de memoria**. Evaluar la eficiencia de los algoritmos es crucial por varias razones:

- **Optimización de recursos:** Permite utilizar los recursos computacionales de manera más efectiva, lo que se traduce en un menor costo y un mejor rendimiento.
- **Escalabilidad:** Ayuda a predecir cómo se comportará un algoritmo a medida que aumenta el tamaño del problema, lo que es fundamental para aplicaciones que deben manejar grandes cantidades de datos.
- **Comparación de algoritmos:** Facilita la comparación de diferentes algoritmos para resolver el mismo problema, lo que permite elegir el más adecuado para una situación específica.

Las principales métricas de eficiencia son:

- **Tiempo:** La cantidad de tiempo que tarda un algoritmo en completar su ejecución. Se mide en unidades de tiempo (segundos, milisegundos, etc.) o en términos de operaciones elementales (comparaciones, asignaciones, etc.).
- **Espacio:** La cantidad de memoria que utiliza un algoritmo durante su ejecución. Se mide en unidades de memoria (bytes, kilobytes, megabytes, etc.).

Existen dos metodologías complementarias para evaluar la eficiencia de un algoritmo:

- **Metodología experimental:** Consiste en implementar el algoritmo y ejecutarlo con diferentes entradas para medir empíricamente su tiempo y espacio de ejecución. Proporciona datos reales pero depende del hardware, lenguaje de programación y otros factores del entorno.
- **Metodología analítica:** Consiste en analizar matemáticamente la estructura del algoritmo para determinar su tiempo y espacio de ejecución en función del tamaño de la entrada, independientemente de factores externos. Proporciona garantías teóricas sobre el comportamiento del algoritmo.

Ambas metodologías son valiosas y se complementan: el análisis teórico guía el diseño y la experimentación valida las predicciones teóricas.

1.2. Descripción de la eficiencia por medio de un modelo analítico

Los modelos de eficiencia se expresan como funciones que relacionan el tamaño de la entrada con la cantidad de recursos consumidos. Estas funciones tienen la forma:

$$T : \mathbb{N} \to \mathbb{R}^+$$

Donde:

- T representa la función de eficiencia (tiempo o espacio).

- $\mathbb{N}$ representa el conjunto de los números naturales, que se utiliza para modelar el tamaño de las entradas.
- $\mathbb{R}^+$ representa el conjunto de los números reales positivos, que se utiliza para modelar la cantidad de recursos consumidos.

La variable independiente de la función T es el tamaño de la entrada, que se denota por n. El tamaño de la entrada puede ser, por ejemplo, el número de elementos en un arreglo, el número de nodos en un grafo, o el número de bits en un número.

La variable dependiente de la función T es la cantidad de tiempo o espacio que consume el algoritmo al procesar una entrada de tamaño n. El tiempo se puede medir en unidades de tiempo (segundos, milisegundos, etc.) o en términos de operaciones elementales (comparaciones, asignaciones, etc.). El espacio se mide en unidades de memoria (bytes, kilobytes, megabytes, etc.).

1.3. Metodología experimental para evaluar la eficiencia

La metodología experimental para evaluar la eficiencia de un algoritmo se basa en los pasos del método científico:

1. **Observaciones (mediciones):** Se implementa el algoritmo y se ejecuta con diferentes entradas de diferentes tamaños. Se miden el tiempo y el espacio de ejecución para cada entrada.
2. **Hipótesis:** Se formula una hipótesis sobre la relación entre el tamaño de la entrada y el tiempo o espacio de ejecución. Por ejemplo, se puede hipotetizar que el tiempo de ejecución es lineal con respecto al tamaño de la entrada.
3. **Predicción:** Se utiliza la hipótesis para predecir el tiempo o espacio de ejecución para nuevas entradas.
4. **Verificación y validación:** Se ejecutan el algoritmo con nuevas entradas y se comparan los resultados medidos con las predicciones. Si los resultados coinciden, la hipótesis se considera verificada y validada. En caso contrario, se debe formular una nueva hipótesis y repetir el proceso.

Es importante tener en cuenta que la metodología experimental puede verse afectada por factores externos, como la carga del sistema, el lenguaje de programación utilizado, y la implementación del algoritmo. Por lo tanto, es importante realizar múltiples mediciones y utilizar técnicas estadísticas para analizar los resultados.

La metodología experimental se puede ayudar de herramientas de *profiling* para medir el tiempo de ejecución y la memoria utilizada por los algoritmos. Además, se puede utilizar herramientas como *benchmarking* para comparar diferentes implementaciones del mismo algoritmo. En la siguiente sección se referencias algunas herramientas para mediciones experimentales y profiling.

1.3.1. Evaluación experimental del tiempo de ejecución

- Python:
  - timeit
  - 5 Ways to Measure Execution Time in Python
- Java:

  - Clase Stopwatch
  - Measure Elapsed Time in Java

- C#:
  - How to Calculate the Code Execution Time in C#?

1.3.2. Evaluación experimental del consumo de memoria

- Python
  - Memory profiling in Python
  - Introduction to Memory Profiling in Python

1.4. Metodología analítica

La metodología analítica proporciona un marco para predecir el rendimiento de un algoritmo mediante el análisis de su estructura, sin depender del hardware ni del entorno de ejecución. Este enfoque se basa en el principio de que el tiempo de ejecución de un programa puede ser expresado como la suma del costo de cada operación elemental multiplicada por su frecuencia de ejecución. En esta sección se desarrollan tres ideas fundamentales que se construyen una sobre la otra: las operaciones elementales y el principio de Knuth, la notación tilde y el orden de crecimiento, y finalmente la notación asintótica formal (O, Ω, Θ).

Operaciones Elementales Las operaciones elementales son aquellas cuyo tiempo de ejecución está acotado superiormente por una constante. Ejemplos comunes incluyen asignaciones, comparaciones, operaciones aritméticas básicas, acceso a elementos de un arreglo y operaciones de salto.

Principio de Knuth El análisis se fundamenta en el principio de Knuth (atribuido al científico de la computación Donald E. Knuth), que establece que el tiempo total de ejecución $T(n)$ puede ser calculado como la suma de los productos del tiempo t_i requerido por cada instrucción y su frecuencia de ejecución f_i:

$$T(n) = \sum_i t_i \cdot f_i$$

Esta igualdad es exacta cuando todas las frecuencias f_i son valores exactos. Cuando alguna f_i es una cota superior (no el valor exacto), la suma proporciona una cota superior del tiempo de ejecución: $T(n) \leq \sum_i t_i \cdot f_i$.

En la práctica, determinar con precisión los valores de t_i puede ser complejo y dependiente de la máquina. Por lo tanto, el análisis a menudo se centra en identificar las operaciones dominantes que tienen el mayor impacto en el tiempo de ejecución.

Notación Tilde La notación tilde, denotada como $T(n) \sim f(n)$, se utiliza para indicar que $f(n)$ es una aproximación asintótica precisa de $T(n)$. Formalmente:

$$T(n) \sim f(n) \quad \text{si y solo si} \quad \lim_{n\to\infty} \frac{T(n)}{f(n)} = 1 \quad (\text{donde } f(n) > 0 \text{ para todo } n \text{ suficientemente grande})$$

En otras palabras, la razón entre $T(n)$ y $f(n)$ tiende a 1 cuando n tiende a infinito. Esto significa que $f(n)$ captura el término dominante de $T(n)$, incluyendo su coeficiente constante, proporcionando una aproximación más precisa que la notación Big-O. A diferencia de Big-O, la notación tilde *preserva* el coeficiente de la función dominante; el siguiente concepto va un paso más allá y lo elimina para centrarse únicamente en la forma funcional.

Orden de Crecimiento El orden de crecimiento simplifica el análisis al enfocarse únicamente en la forma funcional, eliminando las constantes multiplicativas. Si $f(n) \sim a \cdot g(n)$, donde a es una constante positiva, entonces se dice que $f(n)$ tiene un orden de crecimiento de $g(n)$.

El orden de crecimiento permite clasificar algoritmos en categorías amplias (lineal, cuadrático, logarítmico, etc.) y comparar su eficiencia relativa independientemente de factores constantes. Esto es especialmente útil para predecir cómo se comportará un algoritmo cuando el tamaño de la entrada aumenta significativamente. La notación asintótica formal (O, Ω, Θ), que se introduce más adelante, es la herramienta matemática que formaliza estas categorías de crecimiento.

Metodología Simplificada Para análisis rápidos, se puede utilizar una metodología simplificada que se enfoca en la operación dominante:

1. **Identificar la operación crítica:** Localizar la operación elemental más significativa en el ciclo más interno del algoritmo (típicamente: comparaciones, asignaciones, o accesos a memoria).
2. **Contar las ejecuciones:** Determinar cuántas veces se ejecuta esta operación en función del tamaño de la entrada n. Esto generalmente involucra analizar los límites de los ciclos.
3. **Expresar el resultado:** Derivar la función tilde que aproxima el tiempo de ejecución y simplificar al orden de crecimiento correspondiente.

Esta metodología proporciona resultados rápidos y es suficiente para la mayoría de los análisis prácticos.

Ejemplo 1: Determinar T(n) para el siguiente programa

```
double max(double[] a) {
    double x = a[0];                    // t1, f1=1
    for(int i=1; i<a.length; i++)       // t2, f2=1; t3, f3=n; t4, f4=n-1;
        if (a[i]>x) x=a[i];             // t5, f5=n-1; t6, f6 in [0, n-1]
    return x;                           // t7, f7=1
}
```

Listing 1: Encontrar el mayor elemento en un arreglo

Para derivar el tiempo de ejecución, analizamos la frecuencia de cada operación:

- **Inicializaciones** (‘t1‘, ‘t2‘, ‘t7‘): ocurren una sola vez, por lo que $f_1 = f_2 = f_7 = 1$.
- **Evaluación de la condición del ciclo** (‘t3‘): se evalúa n veces. La primera evaluación ocurre cuando ‘i=1‘ y la última cuando ‘i=n‘ (donde falla la condición), dando $f_3 = n$.
- **Cuerpo del ciclo** (‘t4‘, ‘t5‘): se ejecuta $n-1$ veces (para ‘i‘ desde 1 hasta $n-1$), por lo que $f_4 = f_5 = n-1$.

- **Asignación condicional** (‘t6‘): en el peor caso (arreglo ordenado ascendentemente), ocurre $n-1$ veces, con $f_6 \leq n-1$.

Aplicando el principio de Knuth para el peor caso (arreglo ordenado ascendentemente, donde $f_6 = n-1$), la desigualdad se convierte en igualdad:

$$\begin{aligned}
T_{peor}(n) &= t_1 + t_2 + t_3 n + t_4(n-1) + t_5(n-1) + t_6(n-1) + t_7 \\
&= n(t_3 + t_4 + t_5 + t_6) + (t_1 + t_2 - t_4 - t_5 - t_6 + t_7) \\
&\sim an \qquad \text{Función tilde (con } a = t_3 + t_4 + t_5 + t_6\text{)} \\
T_{peor}(n) &= \Theta(n) \qquad \text{Orden de crecimiento: Lineal}
\end{aligned}$$

Ejemplo 2: Determinar T(n) para el siguiente programa

```
int paresCero(int[] a) {
    int conteo=0;                           // t1, f1=1
    for(int i=0; i<a.length; i++)           // t2, f2=1; t3, f3=n+1; t4, f4=n
        for(int j=0; j<a.length; j++)       // t5, f5=n; t6, f6=n*(n+1); t7, f7=n*
    n
            if (i!=j && a[i]+a[j]==0)       // t8, f8=n*n; t9, f9=variable
                conteo++;                   // t10, f10=variable
    return conteo;                          // t11, f11=1
}
```

Listing 2: Encontrar parejas que suman cero

Ejercicio Desarrollar la solución del Ejemplo 2 por la metodología de Knuth. Aplicar el principio de Knuth para calcular $T(n)$ en el peor caso, identificando la operación dominante y su frecuencia de ejecución.

Solución: Aplicando el principio de Knuth, donde el ciclo externo se ejecuta n veces y para cada iteración el ciclo interno se ejecuta n veces, obtenemos que la operación dominante (la comparación) se ejecuta $n \times n = n^2$ veces en el peor caso.

Ejemplo 2 (continuación): Derivación formal mediante la operación elemental representativa.

$$\begin{aligned}
T(n) &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 \\
&= \sum_{i=0}^{n-1} n \\
&= n^2 \\
&\sim n^2 \qquad \text{Función tilde} \\
T(n) &= \Theta(n^2) \qquad \text{Orden de crecimiento: Cuadrático}
\end{aligned}$$

1.4.1. Modelos de costo

Los ejemplos anteriores contabilizaron todas las operaciones con el mismo peso, lo que corresponde al *modelo uniforme*. En el análisis de algoritmos, es útil definir explícitamente un **modelo**

de costo que especifica qué operaciones se contabilizan y cuál es su costo, pues la elección del modelo puede cambiar el resultado del análisis. Diferentes modelos de costo son apropiados para diferentes contextos:

- **Modelo uniforme:** Todas las operaciones elementales tienen el mismo costo.
- **Modelo logarítmico:** El costo de las operaciones depende del tamaño de los operandos.
- **Modelo específico:** Se contabilizan solo ciertas operaciones relevantes (ej: comparaciones, accesos a memoria, operaciones aritméticas).

A continuación se presentan dos ejemplos que ilustran cómo la elección del algoritmo puede afectar drásticamente el rendimiento, incluso cuando ambos resuelven el mismo problema.

Ejemplo 3: Contabilizar sumas, multiplicaciones y accesos al arreglo

```
public static double evalPoly(double[] a, double x)
{
    double s = 0;
    for(int i=0; i<a.length; i++)
        s += a[i]*Math.pow(x,i);
    return s;
}
```

Listing 3: Evaluar un polinomio - siguiendo definición

Ejemplo 4: Contabilizar sumas, multiplicaciones y accesos al arreglo

```
public static double horner(double[] a, double x) {
    double s = a[a.length-1];
    for(int i=a.length-2; i>=0; i--)
        s = s*x + a[i];
    return s;
}
```

Listing 4: Evaluar un polinomio - método de Horner

Comparación de eficiencia: Estos dos ejemplos ilustran una lección fundamental sobre la importancia de elegir el algoritmo correcto:

- **Método directo (‘evalPoly‘):** Sigue la definición matemática literal del polinomio $p(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$. Sin embargo, calcular x^i usando ‘Math.pow(x,i)‘ no es una operación de tiempo constante. Si ‘pow‘ se implementa con multiplicaciones repetidas, su costo es $O(i)$, haciendo que el costo total sea $\sum_{i=0}^{n-1} i = O(n^2)$. Incluso con una implementación más eficiente de ‘pow‘ ($O(\log i)$ por exponenciación binaria), el costo total sería $\sum_{i=0}^{n-1} \log(i) = O(n \log n)$.
- **Método de Horner:** Utiliza la factorización anidada: $p(x) = a_0 + x(a_1 + x(a_2 + \cdots + x(a_{n-2} + x \cdot a_{n-1}) \cdots))$. Realiza exactamente $n-1$ multiplicaciones y $n-1$ sumas, logrando una complejidad de $O(n)$.

Este ejemplo demuestra cómo una reestructuración matemática inteligente puede reducir la complejidad de $O(n^2)$ a $O(n)$, una mejora dramática especialmente notable cuando n es grande.

1.4.2. Notación asintótica

La notación asintótica es una herramienta matemática que nos permite describir el comportamiento de una función cuando su argumento tiende a infinito. En el análisis de algoritmos, utilizamos esta notación para describir el crecimiento del tiempo de ejecución o el uso de memoria.

Notación Big-O (O) La notación Big-O proporciona una cota superior asintótica de la tasa de crecimiento de una función. Formalmente, para funciones $f(n)$ y $g(n)$:

$$f(n) = O(g(n)) \text{ si existen constantes positivas } c \text{ y } n_0 \text{ tales que:}$$
$$0 \leq f(n) \leq c \cdot g(n) \text{ para toda } n \geq n_0$$

Intuitivamente, esto significa que $f(n)$ no crece más rápido que $g(n)$ (salvo por una constante multiplicativa) para entradas suficientemente grandes.

Notación Omega (Ω) La notación Omega proporciona una cota inferior asintótica de la tasa de crecimiento de una función. Formalmente:

$$f(n) = \Omega(g(n)) \text{ si existen constantes positivas } c \text{ y } n_0 \text{ tales que:}$$
$$0 \leq c \cdot g(n) \leq f(n) \text{ para toda } n \geq n_0$$

Esto significa que $f(n)$ crece al menos tan rápido como $g(n)$ para entradas suficientemente grandes.

Notación Theta (Θ) La notación Theta proporciona un límite ajustado asintóticamente. Representa tanto el límite superior como inferior del crecimiento. Una función es $\Theta(g(n))$ si es tanto $O(g(n))$ como $\Omega(g(n))$. Formalmente:

$$f(n) = \Theta(g(n)) \text{ si existen constantes positivas } c_1, c_2 \text{ y } n_0 \text{ tales que:}$$
$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ para toda } n \geq n_0$$

Esto significa que $f(n)$ crece exactamente al mismo ritmo que $g(n)$ (salvo constantes multiplicativas).

Observar que $f(n) = \Theta(g(n))$ si y solo si $f(n) = O(g(n))$ y $f(n) = \Omega(g(n))$

Nota importante: Las notaciones O, Ω y Θ describen cotas sobre la tasa de crecimiento de una *función de costo específica*; no deben confundirse con los conceptos de mejor caso, peor caso o caso promedio, que describen distintos *escenarios de entrada* que pueden producir funciones de costo diferentes. Por ejemplo, para la búsqueda secuencial se puede definir la función del peor caso $T_{peor}(n) = n$, y sobre esa función escribir $T_{peor}(n) = O(n)$, $T_{peor}(n) = \Omega(n)$ y $T_{peor}(n) = \Theta(n)$.

Regla del límite Permite establecer la relación asintótica entre dos funciones $T(n)$ y $f(n)$ según el comportamiento del cociente $T(n)/f(n)$ cuando n tiende a infinito:

$$\lim_{n\to\infty} \frac{T(n)}{f(n)} = \begin{cases} 0 & T(n) \text{ grows slower than } f(n) \implies T(n) = o(f(n)) \\ c > 0 & T(n) \text{ grows at the same rate as } f(n) \implies T(n) = \Theta(f(n)) \\ \infty & T(n) \text{ grows faster than } f(n) \implies T(n) = \omega(f(n)) \end{cases}$$

Aquí o (little-o) y ω (little-omega) son versiones estrictas de O y Ω: si $T(n) = o(f(n))$ entonces también $T(n) = O(f(n))$; si $T(n) = \omega(f(n))$ entonces también $T(n) = \Omega(f(n))$. El caso más útil en la práctica es el límite finito y positivo, que establece Θ directamente.

Conjunto	Nombre
$O(1)$	Constante
$O(\log(n))$	Logarítmico
$O(n)$	Lineal
$O(n\log(n))$	Lineal-logarítmico
$O(n^2)$	Cuadrático
$\vdots$	
$O(n^k)$	Polinómico
$\vdots$	
$O(b^n), b > 1$	Exponencial

Cuadro 1: Jerarquía de conjuntos Big-O. La base del logaritmo no afecta la clase asintótica, ya que $\log_a n = \frac{\log_b n}{\log_b a}$ y el cambio de base introduce solo una constante multiplicativa.

1.5. Análisis de casos

En el análisis de algoritmos, es importante considerar diferentes escenarios de ejecución, ya que el rendimiento puede variar significativamente según las características de la entrada:

- **Mejor caso:** El escenario más favorable para el algoritmo, donde la entrada está organizada de manera óptima. Útil para entender el límite inferior del rendimiento.
- **Peor caso:** El escenario menos favorable, donde la entrada está organizada de la manera más desfavorable. Es el análisis más común en la práctica, ya que proporciona garantías sobre el tiempo máximo de ejecución.
- **Caso promedio:** El desempeño esperado promediando sobre todas las posibles entradas de un tamaño dado, asumiendo una distribución de probabilidad sobre las entradas. Requiere hacer suposiciones sobre la distribución de las entradas.
- **Análisis amortizado:** El desempeño promedio por operación a lo largo de una secuencia de operaciones. Útil cuando algunas operaciones individuales son costosas, pero ocurren con poca frecuencia.

Ejemplo 5: Estimar el tiempo promedio

```
int buscar(T[] datos, T item) {
    for(int i=0; i<datos.length; i++)
        if (item.equals(datos[i]))
            return i;
    return -1;
}
```

Listing 5: Búsqueda secuencial en un arreglo

El siguiente análisis calcula el costo promedio bajo las suposiciones:

- El elemento buscado está presente en el arreglo.
- Cada posición tiene la misma probabilidad $\frac{1}{n}$ de contener el elemento buscado (distribución uniforme).

$$
\begin{aligned}
\overline{T(n)} &= \frac{1}{n}\sum_{i=1}^{n} i \quad \text{Número promedio de comparaciones} \\
&= \frac{1}{n}\left(\frac{n(n+1)}{2}\right) \\
&= \frac{n+1}{2} \\
&\sim \frac{n}{2} \qquad \text{Función tilde} \\
\overline{T(n)} &= \Theta(n) \qquad \text{Orden de crecimiento: Lineal}
\end{aligned}
$$

Es importante considerar también los demás casos:

- **Mejor caso:** El elemento está en la primera posición ($i = 0$), requiriendo una sola comparación, $T_{mejor}(n) = O(1)$.
- **Peor caso:** El elemento está en la última posición o no existe en el arreglo, requiriendo n comparaciones, $T_{peor}(n) = O(n)$.
- **Caso promedio (búsqueda exitosa):** Como se demostró, requiere $\frac{n+1}{2} \approx \frac{n}{2}$ comparaciones, $T_{promedio}(n) = O(n)$.

Nótese que aunque el caso promedio es aproximadamente la mitad del peor caso, ambos tienen el mismo orden de crecimiento lineal $O(n)$.

Ejemplo 6: Analizar el tiempo amortizado[1]

```
private void resize(int capacity) {
    Item[] copy = (Item[]) new Object[capacity];
    for (int i = 0; i < n; i++)
        copy[i] = a[i];
    a = copy;
}

public void add(Item item) {
    if (n == a.length) resize(2*a.length);
    a[n++] = item;
}
```

Listing 6: Cambio de tamaño de un arreglo

Este es un ejemplo clásico de análisis amortizado. Se asume que el arreglo se inicializa con capacidad 1 (en el constructor, no mostrado aquí). Consideremos el comportamiento de la operación `add`:

- **Operación individual:** Una llamada a ‘add‘ puede ser costosa ($O(n)$) cuando el arreglo está lleno y requiere redimensionamiento, o barata ($O(1)$) cuando hay espacio disponible.
- **Frecuencia de redimensionamiento:** Comenzando con capacidad inicial 1 y duplicando cada vez, las redimensionaciones ocurren cuando $n = 1, 2, 4, 8, 16, \ldots$ El costo de copiar elementos es $1 + 2 + 4 + 8 + \cdots + n/2 + n < 2n$.

[1] `ResizingArrayBag.java` de Sedgewick & Wayne

- **Costo total:** Si realizamos N adiciones desde un arreglo vacío, el costo total es la suma de las N inserciones simples más todas las copias durante redimensionamiento, resultando en aproximadamente $3N = O(N)$.
- **Costo amortizado:** El costo promedio por operación es $\frac{O(N)}{N} = O(1)$ constante.

El análisis amortizado es fundamental cuando operaciones costosas ocurren raramente pero de forma predecible. No garantiza que cada operación individual sea rápida, sino que el promedio a largo plazo es eficiente.

1.6. Análisis de espacio

El análisis de espacio evalúa la cantidad de memoria que requiere un algoritmo durante su ejecución. Aunque el tiempo de ejecución suele recibir más atención, el uso eficiente de la memoria es igualmente importante por varias razones:

- **Limitaciones físicas:** La memoria es un recurso finito. Algoritmos con requerimientos excesivos de memoria pueden no ser viables o causar errores por falta de memoria.
- **Rendimiento:** El acceso a memoria afecta el rendimiento. Algoritmos que utilizan menos memoria pueden aprovechar mejor la memoria caché, resultando en ejecución más rápida.
- **Escalabilidad:** El consumo de memoria determina qué tan grandes pueden ser las entradas que el algoritmo puede manejar.

La metodología analítica para el análisis de espacio se basa en un enfoque bottom-up, donde se evalúa el espacio consumido por los componentes básicos y luego se combinan para obtener el espacio total requerido por el algoritmo.

Espacio consumido por datos de tipos primitivos Los tipos de datos primitivos (enteros, booleanos, caracteres, números de punto flotante, etc.) consumen una cantidad fija de memoria que depende del lenguaje de programación y la arquitectura del sistema. Por ejemplo, en Java, un entero ("int") típicamente ocupa 4 bytes, mientras que un número de punto flotante de doble precisión ("double") ocupa 8 bytes[2]. En la tabla 2 se resume el espacio requerido por los tipos de datos primitivos.[3]

Tipo	Java (bytes)	C# (bytes)	Python (bytes)
byte	1	1	
short	2	2	
int	4	4	28 o *más*
long	8	8	
float	4	4	192
double	8	8	-
boolean	1	1	224
char	2	2	8-32

Cuadro 2: Espacio para tipos de datos primitivos

[2]Los tipos punto flotante siguen el estándar IEEE 754
[3]En Python realmente son clases y siguen el modelo de objetos con overhead discutido más abajo

Espacio consumido por referencias Las referencias (o punteros) son variables que almacenan la dirección de memoria de un objeto. El espacio consumido por una referencia también es fijo y depende de la arquitectura del sistema. En sistemas de 64 bits, una referencia típicamente ocupa 8 bytes.

Espacio consumido por objetos sencillos Un objeto sencillo es una instancia de una clase que contiene solo datos de tipos primitivos y/o referencias. El espacio consumido por un objeto sencillo se calcula sumando el espacio consumido por cada uno de sus campos, más un overhead adicional que depende del lenguaje de programación y la implementación de la máquina virtual. En el caso de Java este overhead es de 12 bytes. Adicional a esto, la máquina virtual siempre reserva una cantidad de memoria múltiplo de 8 bytes (64 bits), para asegurar la *alineación* de objetos en memoria. Para completar un múltiplo de 8, se agrega un campo "padding" que no se usa.

Ejemplo Una clase wrapper

```
public class Integer {
    int value;
}
```

Listing 7: Espacio consumido por un objeto Integer

$$\begin{array}{rcrl} Espacio & = & 12 & \text{Overhead} \\ & & +4 & \text{int} \\ \hline & & 16 & \text{Total} \end{array}$$

Ejemplo Python ofrece el metodo `sys.getsizeof` para obtener el espacio asociado a una variable[4]

```
import sys
a=1
b=3.14159
c=True
d="Hello World"          # 11 char
print(sys.getsizeof(a))  # Size of an 'int'  = 28
print(sys.getsizeof(b))  # Size of a 'float' = 24
print(sys.getsizeof(c))  # Size of a 'bool'  = 28
print(sys.getsizeof(d))  # Size of a 'str'   = 60
```

Listing 8: Espacio consumido por distintos tipos de variables en Python

Espacio consumido por objetos con referencias y clases internas Si un objeto contiene referencias a otros objetos, el espacio total consumido incluye el espacio del objeto en sí, más el espacio de todos los objetos a los que hace referencia. Si la clase del objeto contiene clases internas, también se debe considerar el espacio consumido por las instancias de estas clases internas. En Java, una clase interna contiene adicionalmente una referencia hacia la instancia de la clase externa que la contiene.

[4]Es importante notar que 'sys.getsizeof()' puede ser engañoso. Para tipos de datos contenedores como strings, listas o diccionarios, este método devuelve el tamaño del objeto çontenedor.en sí, pero no incluye el tamaño de los elementos que contiene. El uso de memoria real puede ser significativamente mayor.

Ejemplo Una clase wrapper

```
public class List<T> {
    class Node {
        T data;
        Node next;
    }
    Node head;
    int n;
}
```

Listing 9: Espacio consumido por un objeto con clase interna

$Espacio\ Node$	$=$	12	Overhead
		+8	ref data
		+8	ref next
		+8	ref externa
		+4	padding
		40	Total Node
$Espacio\ Lista$	$=$	12	Overhead
		+8	ref head
		+4	int
		24	Total Lista
$Lista(N)$	$=$	24	Instancia de List
		$+40 * N$	Instancias de Node
		$24 + 40 * N$	Total Lista(N)

Espacio consumido por estructuras dinámicas Las estructuras dinámicas, como las listas enlazadas, utilizan memoria de forma dinámica, es decir, la cantidad de memoria que utilizan puede variar durante la ejecución del programa. En el caso de una lista simplemente enlazada, cada nodo de la lista contiene un dato y una referencia al siguiente nodo. Por lo tanto, el espacio total consumido por la lista es proporcional al número de nodos en la lista.

Espacio consumido por arreglos Los arreglos son estructuras de datos fundamentales que almacenan colecciones de elementos del mismo tipo en posiciones de memoria contiguas. El espacio total consumido por un arreglo incluye:

- El overhead del objeto arreglo (típicamente 12 bytes en Java)
- Un campo para almacenar la longitud del arreglo (4 bytes)
- El espacio para todos los elementos del arreglo
- Padding para alineación de memoria (múltiplo de 8 bytes)

El análisis específico depende del tipo de arreglo:

- **Arreglos unidimensionales de tipo primitivo:** El espacio consumido por un arreglo unidimensional de tipo primitivo se calcula multiplicando el tamaño del tipo primitivo por el número de elementos en el arreglo, más el espacio del overhead y la longitud del arreglo.

Ejemplo Un arreglo 1-D de enteros

```
        int[] a = new int[N];

```

Listing 10: Espacio consumido por un arreglo primitivo

$$
\begin{array}{rcll}
E(N) & = & 12 & \text{Overhead} \\
 & & +4 & \text{int length} \\
 & & +4 * N & \text{N int} \\
 & & +0 \text{ ó } +4 & \text{padding} \\
\hline
E(N) & \leq & 16 + 4 * N + 4 & \text{Total peor caso}
\end{array}
$$

- **Arreglos bidimensionales de tipo primitivo:** Un arreglo bidimensional se implementa como un arreglo de arreglos. El arreglo principal (*arreglo de filas*) contiene referencias a los arreglos individuales (*arreglos de columnas*). Por ejemplo, para `int[][] a = new int[m][n]`:
  - Arreglo principal: $12 + 4 + 8m = 16 + 8m$ bytes. Como $8m$ siempre es múltiplo de 8, no se necesita padding.
  - Cada arreglo de columna: $12 + 4 + 4n = 16 + 4n$ bytes. Si n es par, $4n$ es múltiplo de 8 y no hay padding; si n es impar, se agregan 4 bytes de padding, con lo que cada arreglo ocupa $20 + 4n$ bytes.
  - Total (aproximado, despreciando la constante 16 del arreglo principal): $E(m, n) \approx (16 + 8m) + m(16 + 4n) = 16 + 24m + 4mn \approx 24m + 4mn$ bytes.
- **Arreglos de tipo referencia:** Cada elemento es una referencia (8 bytes) a un objeto. El espacio total incluye el overhead del arreglo, las n referencias, más el espacio de todos los objetos referenciados. Por ejemplo, `String[] s = new String[n]` con strings de longitud promedio L requiere aproximadamente $16 + 8n + n \cdot E_{String}(L)$ bytes, donde $E_{String}(L)$ es el espacio de cada objeto `String`.[5]

En resumen, el análisis de espacio es una herramienta fundamental para comprender y optimizar el uso de la memoria en los algoritmos. Al aplicar una metodología analítica bottom-up, es posible prever los requerimientos de memoria y evitar posibles problemas de rendimiento y errores por falta de memoria.

[5] El tamaño exacto de un objeto `String` depende de la versión y configuración de la JVM. En Java 6, el layout incluía campos `offset` y `count` además del arreglo de caracteres, resultando en aproximadamente $60 + 2L$ bytes. En Java 8+, estos campos fueron eliminados, reduciendo el tamaño. Para análisis asintótico, lo relevante es que $E_{String}(L) = \Theta(L)$.